



Collège de
Maisonneuve

ATTESTATION D'ETUDES COLLÉGIALE

CONCEPTION ET PROGRAMMATION DE SITES WEB (NWE.0F)

**Techniques avancées en programmation Web
582-41F-MA**

**TP 1
Création d'une mini-application Node.js API publique
20 %**

**Professeur:
Marcos Sanches
msanches@cmaisonneuve.qc.ca**

**COLLÈGE DE MAISONNEUVE
2030, boul. Pie-IX, bureau 430
Montréal (Québec) H1V 2C8
Téléphone : 514 254-7131**



Création d'une mini-application Node.js avec API publique (20pts)

Félicitations !

Vous avez reçu le mandat de développer une application web complète. Votre mission consiste à créer un site qui permet de récupérer des données depuis une API publique de votre choix, de les enregistrer localement, puis de les afficher dans une page HTML.

Une liste d'API publiques est disponible ici :

<https://github.com/public-apis/public-apis>

Important : Choisissez une API qui vous permet de :

- Faire des requêtes GET dynamiques à partir d'un paramètre (nom, ID, symbole, etc.).
- Fournir à la fois des informations générales et des détails précis.
- Masquer vos clés d'API dans un fichier .env (si nécessaire).

Objectif

Développer une application web à l'aide de **Node.js** et **Express**, capable de :

1. Utiliser une API publique.
2. Sauvegarder les données obtenues dans un fichier .json.
3. Utiliser le fichier .env pour les données sensibles.
4. Afficher ces données dans une interface HTML dynamique stylisée avec CSS.

Consignes techniques

1. Choisir une API publique

- Elle peut concerner la météo, les films, les cryptomonnaies, les pays, les jeux, etc.
- Elle doit permettre de faire une requête paramétrée, comme : GET /endpoint?id=xxx

2. Mettre en place un serveur Express avec deux routes principales

- ♦ `/fetch=:id`
 - Fait une requête à l'API avec l'id (fourni dans l'URL).
 - Enregistre la réponse dans un fichier .json.
 - Redirige ensuite l'utilisateur vers une page HTML (`/view?id=xxx`) pour voir les résultats.
- ♦ `/data/:id`
 - Lit le fichier .json correspondant.
 - Retourne les données en JSON (qui seront utilisées par le frontend).

3. Créer une page HTML dynamique (index.html)

- Récupérer les données via `fetch('/data/:id')`.
- Afficher les informations clés dans le DOM.
- Styliser avec un fichier CSS (design simple, mais propre et lisible).

Livrables attendus

- Un serveur Node.js fonctionnel utilisant Express.
- Deux routes : `/fetch=:id` et `/data/:id`.
- Une interface HTML dynamique affichant les données via JavaScript.
- Un fichier CSS pour la mise en forme.

Remise du projet

- Uploadez votre **documentation au format PDF** sur **LEA – Omnivox**.
- Votre documentation doit inclure :
 - Une **page de garde**
 - Une **brève description du projet**
 - Toutes les **commandes nécessaires à l'installation** des bibliothèques utilisées
 - Les **instructions d'installation et d'exécution**
 - Le **lien vers votre dépôt GitHub**

Présentation individuel orale obligatoire

Après avoir remis votre projet sur LEA – Omnivox, vous devrez **présenter votre application en direct à votre enseignant**.

- Vous devrez **faire une démonstration fonctionnelle** de votre projet.
- Votre enseignant pourra vous poser des questions sur :
 - le code que vous avez écrit,
 - les choix techniques effectués,
 - et votre compréhension générale du fonctionnement de l'application.

Attention :

En cas de présentation incomplète, confuse, ou d'incapacité à expliquer certaines parties du code, une réduction de la note finale pourra être appliquée.

Attention : Tout projet dont la documentation est incomplète se verra attribuer une pénalité de -5 points sur 20.

Barème de Correction (sur 20)

Critère	Points
Présentation de la documentation	1
Lignes de commande (libraires)	1
Qualité des fichiers et structure du serveur Node js	3
REst	2
.env	1
Enregistrer et mettre à jour le fichier JSON	2
Page d'accueil (design – logique – navigation)	3
Page de détails (design – logique – navigation)	3
Intégration de Bootstrap / CSS	1

Qualité du projet rendu – Pas des pages avec d'erreurs, en développement, etc.	1
Architecture du projet / GitHub	1
Qualité du code / commentaires du code	1

Pénalité pour retard.

- Jusqu'à 24 heures : réduction de 1 point sur la note finale sur 20.
- Entre 24 et 48 heures : réduction de 2 points sur la note finale sur 20.
- Après 48 heures de retard, le projet ne serait pas accepté et la note zéro serait accordée.